ODISHA ADARSHA VIDYALAYA SANGATHAN

OAVS TGT Computer Science

Teacher Eligibility Examination — MCQ Question Bank

CHAPTER TOPIC

Unions in C

150	10	16
MCQS	SUB-TOPICS	QUESTION TYPES

Includes Answer Key & Revision-Oriented Explanations for Every Question

Table of Contents

Chapter 1. Concept of Union Collection (15 MCQs, Q1–Q15) Chapter 1 →

Chapter 2. Syntax of Union Declaration and Its Use (15 MCQs, Q16–Q30) Chapter 2 →

Chapter 3. Comparison of Array, Structure and Union (15 MCQs, Q31–Q45) Chapter 3 →

Chapter 4. Array of Unions (15 MCQs, Q46–Q60) Chapter 4 →

Chapter 5. Union as a Member of Structure (15 MCQs, Q61–Q75) Chapter 5 →

Chapter 6. Structure as a Member of Union (15 MCQs, Q76–Q90) Chapter 6 →

Chapter 7. Array as a Member of Union (15 MCQs, Q91–Q105) Chapter 7 →

Chapter 8. Concept of Memory Saving and Union (15 MCQs, Q106–Q120) Chapter 8 →

Chapter 9. Union as a Generic Data Type (15 MCQs, Q121–Q135) Chapter 9 →

Chapter 10. Concept of Anonymous Union (15 MCQs, Q136–Q150) Chapter 10 →

How to use this bank: Each question card shows the question number, question type and topic tag, four options (A–D), the correct answer highlighted in the answer strip, and a short revision-oriented explanation. Correct answers are distributed approximately evenly across options A, B, C and D and are not placed in a predictable pattern.

CHAPTER 1 OF 10 • UNIONS

Concept of Union Collection

This chapter covers the fundamental definition, purpose, characteristics, memory organisation, applications, advantages and limitations of the union collection in C.

Q1 | Basic | Unions DEFINITION

What is a union in C?

A. A collection of functions grouped together

B. A collection of different data types that share the same memory location

C. A type of loop used for iteration

D. A file handling structure

Correct Answer: B

EXPLANATION

A union is a user-defined data type in which all members share the same memory location.

Q2 | Definition-based | Unions DEFINITION

Which statement best defines a union?

A. A derived data type where each member has its own separate memory

B. A primitive data type built into the language

C. A pointer to a structure

D. A derived data type where all members share a common memory location

Correct Answer: D

EXPLANATION

By definition, a union allocates one shared memory block for all its members.

Q3 | Concept-based | Unions CONCEPT

In a union, how many members can hold a valid value at any given time?

A. All members simultaneously

B. Exactly two members at once

C. Only one member at a time

D. None, unions cannot store values

Correct Answer: C

EXPLANATION

Since all members overlap in the same memory, only the most recently assigned member holds a valid value.

Q4 | Syntax-based | Unions SYNTAX

Which keyword is used to declare a union in C?

A. struct
B. class
C. record
D. union

Correct Answer: D

EXPLANATION

The keyword "union" is used to define a union type, similar to how "struct" defines a structure.

Q5 | Statement-based | Unions CHARACTERISTICS

Which of the following statements about unions is TRUE?

A. The size of a union equals the sum of sizes of all its members
B. The size of a union equals the size of its largest member
C. Unions cannot have more than two members
D. Union members cannot be accessed using the dot operator

Correct Answer: B

EXPLANATION

A union's size is determined by its largest member because all members share the same memory space.

Q6 | Assertion-Reason | Unions CONCEPT

Assertion (A): A union can store multiple values of different members simultaneously.
Reason (R): All members of a union share the same memory location.

A. Both A and R are true, R is the correct explanation of A
B. A is false, R is true
C. Both A and R are true, R is not the correct explanation of A
D. Both A and R are false

Correct Answer: B

EXPLANATION

The assertion is false because only one member can hold a valid value at a time; the reason itself is a correct statement.

Q7 | Match-the-Following | Unions

COMPARISON

Match the collection type with its memory behaviour:
I. Union II. Structure III. Array
a. Shared memory for all members b. Separate memory for each member c. Same-type elements

A. I-b, II-a, III-c

B. I-a, II-c, III-b

C. I-c, II-a, III-b

D. I-a, II-b, III-c

Correct Answer: D

EXPLANATION

Union shares memory (a), structure allocates separate memory (b), array holds same-type elements (c).

Q8 | Comparison-based | Unions

COMPARISON

How does a union primarily differ from a structure in terms of memory allocation?

A. A union allocates more memory than a structure

B. A union allocates memory equal to the largest member; a structure allocates memory for all members

C. Both allocate exactly the same fixed memory

D. A structure shares memory like a union

Correct Answer: B

EXPLANATION

This is the core memory difference between unions and structures.

Q9 | Code-snippet | Unions

MEMORY ORGANIZATION

```
union Data {
    int i;
    float f;
    char c;
};
```

What happens to the value of `i` after a value is assigned to `f`?

Union memory layout: **int i** | **float f** | **char c** ← *all members overlap the same block, sized to the largest membe*

A. Both i and f remain valid at the same time

B. The value of i is overwritten; only f is meaningful

C. Only i remains meaningful

D. A compile-time error occurs

Correct Answer: B

EXPLANATION

Since members overlap in memory, assigning to f overwrites the earlier value stored in i.

Q10 | Output-based | Unions

MEMORY ORGANIZATION

```
union U { int x; char y; };
union U u;
u.x = 65;
printf("%d", u.x);
```

What is the output?

A. A

B. 65

C. A garbage value

D. Compilation error

Correct Answer: B

EXPLANATION

Since x was assigned last and read immediately, printing u.x correctly returns 65.

Q11 | Fill-in-the-Blank | Unions

MEMORY ORGANIZATION

The total memory allocated to a union is equal to the size of its _____ member.

A. smallest

B. largest

C. first

D. last

Correct Answer: B

EXPLANATION

A union always reserves memory equal to its largest member to accommodate any member.

Q12 | Scenario-based | Unions APPLICATIONS

A programmer needs a data type that can hold either an integer or a float value, but never both at once, in order to save memory. Which construct is most suitable?

A. Array
B. Structure
C. Union
D. Function

Correct Answer: C

EXPLANATION

A union is ideal here since it stores one value at a time while saving memory compared to a structure.

Q13 | Application-oriented | Unions APPLICATIONS

Unions are commonly used in embedded systems and low-level programming mainly for which purpose?

A. Efficient use of limited memory
B. Increasing code readability
C. Enabling multiple inheritance
D. Simplifying file handling

Correct Answer: A

EXPLANATION

Because unions share memory, they help conserve limited memory resources typical in embedded environments.

Q14 | Previous-Year Pattern | Unions DEFINITION

In C programming, a union is best described as:

A. A data type allowing storage of multiple values simultaneously
B. A type exclusively used for arrays
C. A synonym for structure
D. A memory-efficient data type where members share the same storage

Correct Answer: D

EXPLANATION

This matches the standard exam definition emphasising shared storage among members.

Q15 | Difficult/Tricky | Unions MEMORY ORGANIZATION

A union has members `int i` (4 bytes), `double d` (8 bytes) and `char c` (1 byte). What is the likely size of the union (ignoring padding)?

A. 13 bytes

B. 1 byte

C. 8 bytes

D. 4 bytes

Correct Answer: C

EXPLANATION

The union's size equals the size of its largest member, double (8 bytes), though alignment may add padding.

CHAPTER 2 OF 10 • UNIONS

Syntax of Union Declaration and Its Use

This chapter focuses on the correct syntax for declaring and defining unions, declaring union variables, initialising members, and accessing them, along with valid and invalid declaration patterns.

Q16 | Basic | Unions SYNTAX

Which of the following is a correctly declared union?

A. union { int a; float b; } x;

B. union int a; float b;

C. union x { int a, float b }

D. union x = { int a; float b; }

Correct Answer: A

EXPLANATION

This option follows correct union syntax with a valid member list and variable declaration.

Q17 | Definition-based | Unions SYNTAX

What does the declaration `union Employee { int id; char name[20]; };` represent?

A. A structure named Employee

B. A function named Employee

C. An array of unions

D. A union type named Employee with members id and name

Correct Answer: D

EXPLANATION

This declares a union template named Employee with two members that will share memory.

Q18 | Concept-based | Unions UNION VARIABLES

When declaring a union variable, which of the following is required?

A. Specifying the union tag/type name before the variable name

B. Initializing every member

C. Declaring it inside a function only

D. Using the "struct" keyword

Correct Answer: A

EXPLANATION

A union variable is declared using the union tag/type name, e.g. `union Employee e1;`

Q19 | Syntax-based | Unions ACCESSING MEMBERS

Which is the correct way to access member `id` of union variable `e1` (not a pointer)?

A. e1->id

B. e1::id

C. e1.id

D. e1[id]

Correct Answer: C

EXPLANATION

The dot operator is used to access members of a union variable, exactly as with structures.

Q20 | Statement-based | Unions INITIALIZATION

Which statement about union initialization is TRUE?

A. Only the first member can be initialized in a standard initializer list

B. All members can be initialized simultaneously

C. Initialization is not allowed for unions

D. Only pointer members can be initialized

Correct Answer: A

EXPLANATION

In standard C, a union can be initialized by giving a value to its first member only, e.g. `union U u = {10};`

Q21 | Assertion-Reason | Unions UNION VARIABLES

Assertion (A): `union Data d1, d2;` declares two variables of type union Data. Reason (R): Multiple variables can be declared for the same union tag, separated by commas.

A. Both A and R are true, R is not the correct explanation of A

B. A is true, R is false

C. Both A and R are false

D. Both A and R are true, R is the correct explanation of A

Correct Answer: D

EXPLANATION

Both statements are true, and R correctly explains why the declaration in A is valid.

Q22 | Match-the-Following | Unions

SYNTAX

Match the code with its purpose:

I. `union Tag {...};` **II.** `union Tag t;` **III.** `t.member`

a. Declaring a variable b. Accessing a member c. Defining a union template

A. I-a, II-c, III-b

B. I-b, II-a, III-c

C. I-c, II-b, III-a

D. I-c, II-a, III-b

Correct Answer: D

EXPLANATION

I defines the template, II declares a variable, III accesses a member using the dot operator.

Q23 | Comparison-based | Unions

SYNTAX

How does declaring a union variable differ syntactically from declaring a structure variable?

A. It uses a completely different operator for member access

B. It uses the "union" keyword instead of "struct", but the overall syntax is otherwise similar

C. It cannot be declared inside main()

D. It requires pointers only

Correct Answer: B

EXPLANATION

Union and structure declarations follow a similar pattern, differing mainly in the keyword used.

Q24 | Code-snippet | Unions

INITIALIZATION

```
union Item {
    int code;
    float price;
};
union Item it = {101};
```

What is stored in `it` after this initialization?

A. Both code and price are set

B. price is initialized to 101.0

C. A compilation error occurs

D. Only code is initialized, to 101

Correct Answer: D

EXPLANATION

When only one initializer is given, it is assigned to the first declared member, here code.

Q25 | Output-based | Unions

ACCESSING MEMBERS

```
union U { int a; char b; };
union U u;
u.a = 320;
printf("%d", u.b);
```

What is the likely nature of the output?

A. Always exactly 320

B. Always 0

C. Compilation error

D. An implementation-defined value, because a and b overlap in memory

Correct Answer: D

EXPLANATION

Since b overlaps only part of a's memory, reading b after writing a gives an implementation-dependent byte value.

Q26 | Fill-in-the-Blank | Unions

UNION VARIABLES

To declare a union variable named `r` using the tag `Record`, the correct syntax is: `union Record _____;`

A. Record r

B. union r

C. new Record()

D. r

Correct Answer: D

EXPLANATION

The correct declaration is `union Record r;`, so the blank is filled with the variable name "r".

Q27 | Scenario-based | Unions

VALID AND INVALID DECLARATIONS

A student writes `union Data { int i; float f; }; Data d;` and gets a compiler error. What is the mistake?

A. The "union" keyword is missing before the variable declaration

B. A union cannot contain both int and float

C. A semicolon is missing after the union body

D. Member names cannot be i and f

Correct Answer: A

EXPLANATION

In standard C, the variable declaration must use `union Data d;`, not just `Data d;`, unless typedef is used.

Q28 | Application-oriented | Unions — VALID AND INVALID DECLARATIONS

Which of the following is an INVALID union declaration?

A. union Point { int x; int y; };

B. union Point p; int x; float y; };

C. union { int x; int y; } p;

D. union Point { int x; int y; } p1, p2;

Correct Answer: B

EXPLANATION

This option has mismatched braces and member declarations outside the union body, making it invalid syntax.

Q29 | Previous-Year Pattern | Unions — ACCESSING MEMBERS

Which operator is used to access a union member through a pointer to the union?

A. .

B. ::

C. &

D. ->

Correct Answer: D

EXPLANATION

The arrow operator (->) is used to access members through a pointer, exactly as with structures.

Q30 | Difficult/Tricky | Unions — INITIALIZATION

Consider `union U { int a; char b[4]; } u = {65};`. What does `u.b[0]` most likely contain on a little-endian system?

A. 0

B. An unrelated value

C. 65

D. Compilation error

Correct Answer: C

EXPLANATION

On little-endian systems, the least significant byte of a is stored first, so b[0] holds 65.

CHAPTER 3 OF 10 • UNIONS

Comparison of Array, Structure and Union

This chapter compares arrays, structures and unions in terms of similarities, differences, memory allocation, storage characteristics, data organisation, and appropriate use cases.

Q31 | Basic | Unions

COMPARISON

Which of these can store elements of different data types?

A. Array only

B. Structure and Union

C. Array and Structure

D. None of these

Correct Answer: B

EXPLANATION

Arrays store elements of the same type, while structures and unions can hold members of different types.

Q32 | Definition-based | Unions

COMPARISON

Which statement correctly differentiates arrays from structures/unions?

A. Arrays store heterogeneous data; structures store homogeneous data

B. Arrays store homogeneous data; structures/unions can store heterogeneous data

C. Arrays and structures are identical in nature

D. Unions cannot store any combination of data types

Correct Answer: B

EXPLANATION

This is the fundamental distinction: arrays are homogeneous, while structures and unions are heterogeneous.

Q33 | Concept-based | Unions

MEMORY ALLOCATION

Which of these allocates memory equal to the sum of all member sizes?

Union memory layout: **int i** **float f** **char c** ← *all members overlap the same block, sized to the largest membe*

A. Union

B. Structure

C. Both union and structure

D. Neither

Correct Answer: B

EXPLANATION

A structure allocates separate memory for each member, so its total size is the sum of member sizes.

Q34 | Syntax-based | Unions COMPARISON

Which keyword differentiates a structure declaration from a union declaration?

A. array vs struct
B. int vs float
C. struct vs record
D. struct vs union

Correct Answer: D

EXPLANATION

The declaration syntax is identical except for using "struct" versus "union" as the keyword.

Q35 | Statement-based | Unions MEMORY USAGE

Which statement is TRUE regarding memory usage?

A. Union memory usage is always greater than structure memory usage
B. Array and union always use equal memory
C. Structure always uses less memory than union
D. Structure memory usage is generally greater than or equal to union memory usage

Correct Answer: D

EXPLANATION

Since structures store all members separately, they generally use equal or more memory than a union with the same members.

Q36 | Assertion-Reason | Unions MEMORY ALLOCATION

Assertion (A): A union is more memory-efficient than a structure with the same members.
Reason (R): A union allocates memory only for its largest member.

A. Both A and R are true, R is not the correct explanation of A
B. A is false, R is true
C. Both A and R are true, R is the correct explanation of A
D. Both A and R are false

Correct Answer: C

EXPLANATION

Both statements are true, and R correctly explains why unions save memory compared to structures.

Q37 | Match-the-Following | Unions

DATA ORGANIZATION

Match the collection with its data organisation:
I. Array II. Structure III. Union
a. Same data type elements b. Different types, separate memory c. Different types, shared memory

A. I-b, II-c, III-a

B. I-c, II-a, III-b

C. I-a, II-c, III-b

D. I-a, II-b, III-c

Correct Answer: D

EXPLANATION

Array holds same-type elements; structure allocates separate memory per member; union shares memory.

Q38 | Comparison-based | Unions

SIMILARITIES

Which of the following is a similarity between structures and unions?

A. Both allocate memory equal to the sum of member sizes

B. Both cannot contain arrays as members

C. Both can contain members of different data types and are accessed using the dot/arrow operator

D. Both are indexed like arrays

Correct Answer: C

EXPLANATION

Structures and unions both allow heterogeneous members that are accessed via similar operators.

Q39 | Code-snippet | Unions

MEMORY USAGE

```
struct S { int a; float b; };
union U { int a; float b; };
```

Which statement about `sizeof(struct S)` and `sizeof(union U)` is generally TRUE?

A. sizeof(struct S) is generally less than sizeof(union U)

B. Both are always exactly equal

C. sizeof(union U) is always 0

D. sizeof(struct S) is generally greater than or equal to sizeof(union U)

Correct Answer: D

EXPLANATION

Since a structure allocates memory for all members while a union shares memory, the structure size is typically larger.

Q40 | Output-based | Unions MEMORY ALLOCATION

For `struct S {char c; int i;};` and `union U {char c; int i;};` on a typical system, which generally has the smaller `sizeof` value?

A. struct S
B. union U
C. Both are equal
D. Cannot be determined at all

Correct Answer: B

EXPLANATION
The union shares memory for its largest member, making it smaller than or equal to the structure's size.

Q41 | Fill-in-the-Blank | Unions DATA ORGANIZATION

An array stores elements of the _____ data type, whereas structures and unions can store elements of _____ data types.

A. different, same
B. same, same
C. same, different
D. different, different

Correct Answer: C

EXPLANATION
Arrays are homogeneous collections; structures and unions are heterogeneous collections.

Q42 | Scenario-based | Unions APPROPRIATE USE CASES

A developer needs to store a student's roll number, name and marks simultaneously, with all fields accessible at once. Which is most appropriate?

A. Union
B. Array of int
C. Structure
D. A single variable

Correct Answer: C

EXPLANATION
Since all fields must be accessible simultaneously, a structure is appropriate; a union would overwrite earlier data.

Q43 | Application-oriented | Unions APPROPRIATE USE CASES

Which data structure is best suited when only one of several possible data types needs to be stored at a time, to conserve memory?

A. Array

B. Structure

C. Union

D. Linked list

Correct Answer: C

EXPLANATION

A union is designed for exactly this use case: storing one of many possible types efficiently.

Q44 | Previous-Year Pattern | Unions MEMORY USAGE

Which of the following correctly compares memory usage for the same set of members?

A. Union size is generally less than or equal to structure size

B. Structure size is always less than union size

C. Array size equals union size always

D. Union size is always greater than structure size

Correct Answer: A

EXPLANATION

A union's memory is generally less than or equal to a structure holding the same members.

Q45 | Difficult/Tricky | Unions COMPARISON

A structure and a union both contain the same three members: char, int and double. Which statement is MOST accurate about their sizes (ignoring padding)?

A. Both always have the same size

B. Union size equals the sum of all member sizes

C. Structure size equals the sum of all member sizes; union size equals the size of the largest member (double)

D. Structure size equals the size of the largest member

Correct Answer: C

EXPLANATION

This is the defining memory allocation rule distinguishing structures from unions.

CHAPTER 4 OF 10 • UNIONS

Array of Unions

This chapter covers declaring, initialising, and accessing arrays of union elements, along with their characteristics, memory concepts and applications.

Q46 | Basic | Unions

DECLARATION

What is an "array of unions"?

A. A union containing an array as a member

B. An array containing structures

C. A collection of multiple union variables of the same union type

D. A pointer to a union

Correct Answer: C

EXPLANATION

An array of unions is a collection where each element is a variable of the same union type.

Q47 | Definition-based | Unions

DECLARATION

Which declaration correctly defines an array of 5 unions of type `union Data`?

A. union Data[5] arr;

B. union Data arr[5];

C. array union Data arr(5);

D. Data union arr[5];

Correct Answer: B

EXPLANATION

The syntax follows a normal array declaration: type name followed by array name and size in brackets.

Q48 | Concept-based | Unions

MEMORY CONCEPTS

In an array of unions, how is the total memory of the array calculated?



A. Sum of sizes of all members multiplied by number of elements

B. Size of the smallest member only

C. It is always a fixed 100 bytes

D. Size of one union element multiplied by the number of elements

Correct Answer: D

EXPLANATION

Just like any array, total memory equals the size of one union element multiplied by the array length.

Q49 | Syntax-based | Unions ACCESSING ELEMENTS

How do you access the `id` member of the 3rd element (index 2) in `union Data arr[5];`?

A. arr(2).id

B. arr.id[2]

C. arr[2].id

D. arr->id[2]

Correct Answer: C

EXPLANATION

Standard indexing followed by the dot operator is used to access a member of a specific array element.

Q50 | Statement-based | Unions CHARACTERISTICS

Which statement about arrays of unions is TRUE?

A. All elements must hold values in the same member simultaneously

B. Each element can independently hold a value in any one of its members

C. An array of unions is not allowed in C

D. Elements cannot be accessed using indices

Correct Answer: B

EXPLANATION

Each union element in the array independently manages its own shared memory for its members.

Q51 | Assertion-Reason | Unions APPLICATIONS

Assertion (A): An array of unions can be used to store a collection of variant records.
Reason (R): Each element of the array behaves as an independent union instance.

A. Both A and R are true, R is not the correct explanation of A

B. A is true, R is false

C. Both A and R are false

D. Both A and R are true, R is the correct explanation of A

Correct Answer: D

EXPLANATION

Both are true, and R correctly explains why arrays of unions can hold multiple independent variant values.

Q52 | Match-the-Following | Unions

APPLICATIONS

Match the code with its purpose:

I. `union Data arr[3];` **II.** `arr[0].x = 5;` **III.** `sizeof(arr)`

a. Accessing member of first element b. Declaring an array of 3 unions c. Total memory of the array

A. I-a, II-b, III-c

B. I-c, II-b, III-a

C. I-b, II-a, III-c

D. I-b, II-c, III-a

Correct Answer: C

EXPLANATION

I declares the array, II accesses a member of an element, III gives total memory size.

Q53 | Comparison-based | Unions

CHARACTERISTICS

How does an array of unions differ from a single union variable?

A. It behaves identically since unions cannot be arrayed

B. It cannot store different member values per element

C. It provides multiple independent union instances instead of just one

D. It always uses less total memory than a single union

Correct Answer: C

EXPLANATION

An array of unions provides multiple slots, each independently able to store a different member's value.

Q54 | Code-snippet | Unions

ACCESSING ELEMENTS

```
union U { int x; float y; };
union U arr[3];
arr[0].x = 10;
arr[1].y = 2.5;
printf("%d", arr[0].x);
```

What is the output?

A. 10

B. 2 (truncated float)

C. A garbage value

D. Compilation error

Correct Answer: A

EXPLANATION

Since arr[0].x was directly assigned, printing it correctly returns 10.

Q55 | Output-based | Unions

INITIALIZATION

```
union U { char c; int i; };
union U arr[2] = { {'A'}, {66} };
printf("%c", arr[0].c);
```

What is the output?

A. 66

B. B

C. Compilation error

D. A

Correct Answer: D

EXPLANATION

The first element's member c was initialized with 'A', so printing it with %c gives 'A'.

Q56 | Fill-in-the-Blank | Unions

DECLARATION

To declare an array of 10 union elements of type `union Record`, the syntax is: `union Record _____;`

A. arr[10]

B. arr(10)

C. arr{10}

D. arr::10

Correct Answer: A

EXPLANATION

Standard array declaration syntax uses square brackets to specify the number of elements.

Q57 | Scenario-based | Unions

APPLICATIONS

A program must store 50 sensor readings, where each reading could be either an integer or a floating-point value depending on the sensor type. Which is most suitable?

A. An array of unions, one union per sensor reading

B. A single union variable for all readings

C. A 2D array of int only

D. A single structure

Correct Answer: A

EXPLANATION

An array of unions lets each of the 50 readings independently store either type efficiently.

Q58 | Application-oriented | Unions APPLICATIONS

Arrays of unions are useful in applications such as:

A. Storing variant data records, like a table of tagged union entries

B. Sorting strings alphabetically

C. Implementing recursion

D. File compression algorithms only

Correct Answer: A

EXPLANATION

Arrays of unions are commonly used to store collections of variant/tagged data efficiently.

Q59 | Previous-Year Pattern | Unions MEMORY CONCEPTS

What is the total memory occupied by `union U arr[4];` if `union U` occupies 8 bytes?

A. 8 bytes

B. 32 bytes

C. 16 bytes

D. 4 bytes

Correct Answer: B

EXPLANATION

Total memory = size of one union element (8 bytes) x number of elements (4) = 32 bytes.

Q60 | Difficult/Tricky | Unions MEMORY CONCEPTS

In `union U { int a; char b[4]; } arr[3];`, if only `arr[1].a` is assigned a value, what happens to `arr[0]` and `arr[2]`?

A. They automatically get the same value as arr[1]

B. Their members remain uninitialized/ indeterminate unless separately assigned

C. They cause a compilation error

D. They become read-only

Correct Answer: B

EXPLANATION

Each array element is an independent union instance; assigning one element does not affect the others.

CHAPTER 5 OF 10 • UNIONS

Union as a Member of Structure

This chapter covers declaring, initialising and accessing a union that has been embedded as a member of a structure, along with its applications.

Q61 | Basic | Unions DECLARATION

What does it mean when a union is used as a member of a structure?

A. One of the structure's members is a union type, allowing that member to hold one of several types

B. The structure becomes a union

C. It is not allowed in C

D. The union replaces all structure members

Correct Answer: A

EXPLANATION

A union can be embedded inside a structure as one of its fields, giving that field variant behaviour.

Q62 | Definition-based | Unions DECLARATION

Which of the following correctly declares a union as a member of a structure?

A. union S { struct U u; };

B. struct S { union U u; int type; };

C. struct union S { int type; };

D. struct S union U u;

Correct Answer: B

EXPLANATION

This correctly embeds a union variable u as a member inside structure S.

Q63 | Concept-based | Unions PRACTICAL USAGE

Why is a union often embedded in a structure along with a "type" or "tag" field?

A. To track which member of the union is currently valid

B. To increase the union's memory size

C. To convert the union into an array

D. To disable member access

Correct Answer: A

EXPLANATION

A separate tag field helps the programmer know which union member currently holds valid data, forming a "tagged union".

Q64 | Syntax-based | Unions

MEMBER ACCESS

Given `struct S { int type; union U u; } s;`, how do you access member `x` of the union inside `s`?

A. s.x.u

B. s->u->x

C. u.s.x

D. s.u.x

Correct Answer: D

EXPLANATION

You first access the structure member u, then its union member x, using chained dot operators.

Q65 | Statement-based | Unions

APPLICATIONS

Which statement is TRUE about a structure containing a union member?

A. The structure's total size ignores the union entirely

B. The structure's total size includes space for the union member (sized as the union's largest member) plus its other members

C. Structures cannot contain unions

D. The union member must always be the first member

Correct Answer: B

EXPLANATION

The structure allocates space for all its members, including the union, which itself occupies space equal to its largest member.

Q66 | Assertion-Reason | Unions

APPLICATIONS

Assertion (A): Embedding a union in a structure with a tag field is a common technique to implement variant records.
Reason (R): The tag field indicates which union member is currently active.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are false

D. Both A and R are true, R is the correct explanation of A

Correct Answer: D

EXPLANATION

Both are true, and R explains the purpose of the tag field in this pattern.

Q67 | Match-the-Following | Unions

DECLARATION

Match the item with its role:

I. `struct S { union U u; };` **II.** `s.u.member` **III. tag field**

a. Access union member inside structure **b. Declares union as structure member**

c. Tracks active union member

A. I-a, II-c, III-b

B. I-b, II-a, III-c

C. I-c, II-b, III-a

D. I-b, II-c, III-a

Correct Answer: B

EXPLANATION

I declares the embedding, II accesses the nested member, III is the tag-tracking pattern.

Q68 | Comparison-based | Unions

PRACTICAL USAGE

How does embedding a union in a structure differ from embedding a structure in a union?

A. They are functionally identical in every respect

B. Neither is allowed in standard C

C. Union-in-structure gives each structure instance its own variant field with tag tracking; structure-in-union shares the whole structure's memory with other union members

D. Union-in-structure always uses more memory than structure-in-union

Correct Answer: C

EXPLANATION

The two nesting patterns serve different purposes: one embeds a variant field, the other overlaps a whole structure with other members.

Q69 | Code-snippet | Unions

MEMBER ACCESS

```
struct Shape {
    int type;
    union { float radius; float side; } dim;
};
struct Shape s;
s.type = 1;
s.dim.radius = 5.0;
printf("%.2f", s.dim.radius);
```

What is the output?

A. 0.00

B. A garbage value

C. Compilation error

D. 5.00

Correct Answer: D

EXPLANATION

The value directly assigned to s.dim.radius is correctly printed as 5.00.

Q70 | Output-based | Unions

MEMBER ACCESS

```
struct Data { int type; union { int i; char c; } val; };
struct Data d;
d.val.c = 'Z';
printf("%c", d.val.c);
```

What is the output?

A. 90

B. A garbage value

C. Z

D. Compilation error

Correct Answer: C

EXPLANATION

Since d.val.c was directly assigned 'Z', printing it with %c outputs 'Z'.

Q71 | Fill-in-the-Blank | Unions PRACTICAL USAGE

In a tagged union design, the structure field that indicates which union member is active is commonly called the _____ field.

A. type/tag
B. index
C. pointer
D. size

Correct Answer: A

EXPLANATION

This field is conventionally called the "type" or "tag" field in tagged union implementations.

Q72 | Scenario-based | Unions APPLICATIONS

A shape-processing program needs to store either a circle's radius or a rectangle's width/height, plus a type indicator, in one record. Which design fits best?

A. A plain union without any tag field
B. Two completely unrelated separate variables
C. A structure containing a type field and a union of the shape-specific data
D. An array of floats only

Correct Answer: C

EXPLANATION

This is the classic tagged-union pattern: a structure with a tag field and an embedded union for variant data.

Q73 | Application-oriented | Unions APPLICATIONS

Structures containing unions are widely used in:

A. Implementing variant/tagged data types, such as in compilers and parsers
B. String comparison functions
C. Sorting algorithms
D. File I/O operations only

Correct Answer: A

EXPLANATION

This nesting pattern is a standard technique for implementing variant types in real-world software like compilers.

Q74 | Previous-Year Pattern | Unions

SYNTAX

When a union is a member of a structure, the union's contribution to the structure's overall size is:

A. The size of the union's largest member

B. The sum of all union member sizes

C. Always zero

D. Twice the largest member's size

Correct Answer: A

EXPLANATION

The union member still follows union rules internally, contributing only its largest member's size.

Q75 | Difficult/Tricky | Unions

SYNTAX

```
struct Rec {
    int type;
    union { int i; double d; } value;
};
```

If `sizeof(int)=4` and `sizeof(double)=8`, what is the union's contribution to `struct Rec`'s size (ignoring alignment)?

A. 8 bytes

B. 4 bytes

C. 12 bytes

D. 16 bytes

Correct Answer: A

EXPLANATION

The union occupies space equal to its largest member (double, 8 bytes), regardless of the other member's size.

CHAPTER 6 OF 10 • UNIONS

Structure as a Member of Union

This chapter covers declaring, initialising and accessing a structure that has been embedded as a member of a union, along with its applications and characteristics.

Q76 | Basic | Unions

DECLARATION

What does it mean when a structure is used as a member of a union?

A. The structure becomes a pointer

B. It is not allowed in C

C. The structure replaces the union entirely

D. The union contains a structure as one of its members, sharing memory with other union members

Correct Answer: D

EXPLANATION

A structure can be one of the members of a union, and it shares the union's common memory space with other members.

Q77 | Definition-based | Unions

DECLARATION

Which declaration correctly places a structure as a member of a union?

A. struct U { union S s; };

B. union struct U { int a; };

C. union U { struct { int a; int b; } s; int x; };

D. struct union U s;

Correct Answer: C

EXPLANATION

This correctly declares a union U with a structure member s alongside another member x.

Q78 | Concept-based | Unions

CHARACTERISTICS

When a union contains a structure member, how much memory does that structure member require within the union?

A. Only the size of the structure's largest field

B. The full size of the structure (sum of its own fields, with alignment)

C. Zero bytes

D. The size of an int only

Correct Answer: B

EXPLANATION

The structure itself is treated as a single member, requiring memory equal to its own total size.

Q79 | Syntax-based | Unions ACCESSING MEMBERS

Given `union U { struct { int a; int b; } s; int x; } u;`, how do you access member `a`?

A. u.a

B. u->s->a

C. s.u.a

D. u.s.a

Correct Answer: D

EXPLANATION

You access the structure member first (s), then its field (a), using chained dot operators.

Q80 | Statement-based | Unions CHARACTERISTICS

Which statement is TRUE regarding a union with a structure member?

A. The union's total size is the sum of the structure's size and other members' sizes

B. Structures cannot be union members

C. The union's total size equals the size of its largest member, which could be the structure

D. The structure member cannot have more than one field

Correct Answer: C

EXPLANATION

The union follows normal rules; its size equals the size of the largest member, which may be the embedded structure.

Q81 | Assertion-Reason | Unions CHARACTERISTICS

Assertion (A): A structure member inside a union can overlap in memory with other union members.
Reason (R): All members of a union share a common memory location.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are true, R is the correct explanation of A

D. Both A and R are false

Correct Answer: C

EXPLANATION

Both statements are true, and R correctly explains why the structure member overlaps with others.

Q82 | Match-the-Following | Unions

DECLARATION

Match the item with its role:

I. `union U { struct S s; int x; };` **II.** `u.s.field` **III.** `sizeof(union U)`

a. Equals size of the largest member b. Declares structure inside union c. Accesses structure field

A. I-a, II-b, III-c

B. I-c, II-a, III-b

C. I-b, II-a, III-c

D. I-b, II-c, III-a

Correct Answer: D

EXPLANATION

I declares the nested structure, II accesses its field, III gives the union's total size rule.

Q83 | Comparison-based | Unions

APPLICATIONS

What is a key difference between "structure inside union" and "union inside structure"?

A. They behave in exactly the same way

B. Neither pattern is valid in standard C

C. In union-of-structure, the structure shares memory with sibling union members; in structure-of-union, the union is one independently-sized field among the structure's separately allocated members

D. Structure-inside-union always uses less memory

Correct Answer: C

EXPLANATION

The memory-sharing behaviour differs depending on which container is the outer type.

Q84 | Code-snippet | Unions

ACCESSING MEMBERS

```
union U {
    struct { int x; int y; } point;
    int z;
};
union U u;
u.point.x = 3;
u.point.y = 4;
printf("%d %d", u.point.x, u.point.y);
```

What is the output?

A. Garbage values

B. 0 0

C. 3 4

D. Compilation error

Correct Answer: C

EXPLANATION

Both fields of the structure member were directly assigned and remain valid since z was never written.

Q85 | Output-based | Unions

MEMORY BEHAVIOUR

```
union U { struct { int a; int b; } s; long l; };
union U u;
u.l = 100;
printf("%d", u.s.a);
```

What best describes the output?

Union memory layout: **int i** **float f** **char c** ← *all members overlap the same block, sized to the largest membe*

A. It is always exactly 100

B. It is always 0

C. It is implementation-defined, since writing to l overlaps with s's memory

D. Compilation error

Correct Answer: C

EXPLANATION

Writing to l overwrites the shared memory used by s, so reading s.a afterward gives an implementation-dependent value.

Q86 | Fill-in-the-Blank | Unions CHARACTERISTICS

When a structure is a member of a union, the union's overall size is determined by comparing the structure's size with the sizes of _____.

A. only the first member

B. only pointer members

C. all other union members

D. global variables

Correct Answer: C

EXPLANATION

The union's size is the maximum among all its members' sizes, including the structure member.

Q87 | Scenario-based | Unions APPLICATIONS

A network protocol handler needs a union where one interpretation of raw bytes is a structure of header fields, and another interpretation is a single integer. Which design fits?

A. Two completely separate unrelated structures

B. An array of chars only

C. A structure containing two unions

D. A union containing both a structure of header fields and an int member

Correct Answer: D

EXPLANATION

This is a classic technique for reinterpreting raw memory as different types, useful in protocol-handling code.

Q88 | Application-oriented | Unions APPLICATIONS

Structures nested inside unions are useful for:

A. Byte-level reinterpretation of data, such as splitting an integer into component bytes/fields

B. Implementing linked lists only

C. String concatenation

D. File compression exclusively

Correct Answer: A

EXPLANATION

This pattern is often used to reinterpret data at the byte or field level, common in systems programming.

Q89 | Previous-Year Pattern | Unions CHARACTERISTICS

If a union contains a structure member of size 8 bytes and an int member of size 4 bytes, the union's size will be:

A. 12 bytes

B. 4 bytes

C. 8 bytes (or larger due to alignment)

D. 0 bytes

Correct Answer: C

EXPLANATION

The union's size equals its largest member, which here is the 8-byte structure.

Q90 | Difficult/Tricky | Unions CHARACTERISTICS

In `union U { struct { char a; int b; } s; double d; };`, assuming the inner structure requires 8 bytes (with padding) and double requires 8 bytes, what is the union's total size?

A. 16 bytes

B. 12 bytes

C. 8 bytes

D. 4 bytes

Correct Answer: C

EXPLANATION

Since both the structure (with padding) and double occupy 8 bytes, the union's size remains 8 bytes, the largest among equal-sized members.

CHAPTER 7 OF 10 • UNIONS

Array as a Member of Union

This chapter covers declaring, initialising and accessing an array that has been embedded as a member of a union, along with its applications and memory behaviour.

Q91 | Basic | Unions

DECLARATION

What does it mean when an array is used as a member of a union?

A. The array becomes the union's tag

B. It is not allowed in C

C. Arrays cannot be union members under any circumstances

D. One of the union's members is an array, sharing memory with the union's other members

Correct Answer: D

EXPLANATION

A union can have an array as one of its members, and that array shares the same memory as other members.

Q92 | Definition-based | Unions

DECLARATION

Which declaration correctly places an array as a member of a union?

A. union U[4] { int i; };

B. array union U { int i; };

C. union U { int i; char bytes[4]; };

D. union U { array char bytes; };

Correct Answer: C

EXPLANATION

This correctly declares a union with an int member and a 4-element char array member.

Q93 | Concept-based | Unions

APPLICATIONS

Why is an array member (like `char bytes[4]`) commonly used alongside an `int` member in a union?

A. To allow byte-level access to the individual bytes of the int value

B. To increase the union's overall memory usage

C. To disable the int member permanently

D. To convert the union into a structure

Correct Answer: A

EXPLANATION

This common technique lets programmers inspect or manipulate individual bytes of a larger data type.

Q94 | Syntax-based | Unions

ACCESS

Given `union U { int i; char b[4]; } u;`, how do you access the second byte of the array member?

A. u.b(1)

B. u.b[1]

C. u[1].b

D. u->b[1]

Correct Answer: B

EXPLANATION

Standard array indexing combined with the dot operator is used to access a specific element of the array member.

Q95 | Statement-based | Unions

MEMORY BEHAVIOUR

Which statement is TRUE about an array member inside a union?

Union memory layout: **int i** **float f** **char c** ← *all members overlap the same block, sized to the largest membe*

A. Array members are ignored when calculating the union's size

B. The array's total byte size contributes to determining the union's overall size along with other members

C. An array member always makes the union invalid

D. Arrays inside unions cannot be indexed

Correct Answer: B

EXPLANATION

The array's size is one of the sizes considered when the compiler determines the union's overall size (the maximum).

Q96 | Assertion-Reason | Unions

APPLICATIONS

Assertion (A): A char array member in a union is often used to examine the byte representation of another member, such as an int.
Reason (R): All members of a union overlap in the same memory location.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are true, R is the correct explanation of A

D. Both A and R are false

Correct Answer: C

EXPLANATION

Both statements are true, and R explains why byte-level inspection works via overlapping memory.

Q97 | Match-the-Following | Unions

DECLARATION

Match the item with its role:

I. `union U { int i; char b[4]; };` **II.** `u.b[0]` **III.** `sizeof(union U)`

a. Equals size of largest member b. Declares array member in union c. Accesses first byte

A. I-a, II-b, III-c

B. I-c, II-a, III-b

C. I-b, II-c, III-a

D. I-b, II-a, III-c

Correct Answer: C

EXPLANATION

I declares the union with an array member, II accesses an element, III applies the standard size rule.

Q98 | Comparison-based | Unions

APPLICATIONS

How does an array member in a union differ from an array of unions?

A. An array member is one field inside a single union sharing memory with other members; an array of unions is multiple independent union instances

B. They are exactly the same concept

C. Arrays cannot be union members at all

D. An array of unions always uses less memory

Correct Answer: A

EXPLANATION

These are distinct concepts: one nests an array inside a union, the other creates multiple union instances.

Q99 | Code-snippet | Unions

ACCESS

```
union U { int i; char b[4]; };
union U u;
u.i = 1;
printf("%d", u.b[0]);
```

On a little-endian system, what is the likely output?

A. 0

B. 4

C. Compilation error

D. 1

Correct Answer: D

EXPLANATION

On little-endian systems, the least significant byte of i (value 1) is stored first, so b[0] is 1.

Q100 | Output-based | Unions

ACCESS

```
union U { int i; char b[4]; };
union U u;
u.b[0] = 'A';
u.b[1] = 'B';
printf("%c%c", u.b[0], u.b[1]);
```

What is the output?

A. AB

B. BA

C. Garbage characters

D. Compilation error

Correct Answer: A

EXPLANATION

Both array elements were directly assigned characters, so printing them returns "AB" as expected.

Q101 | Fill-in-the-Blank | Unions

MEMORY BEHAVIOUR

When an array is a member of a union, the union's total size is at least as large as the _____ of the array member.

A. first element

B. average size

C. index count only, ignoring element size

D. total byte size

Correct Answer: D

EXPLANATION

The array's full byte size (elements times element size) is considered when determining the union's overall size.

Q102 | Scenario-based | Unions

APPLICATIONS

A programmer wants to inspect individual bytes of a 4-byte integer to check the system's endianness. Which union design helps achieve this?

A. union { int i; char b[4]; }

B. struct { int i; char b[4]; }

C. int arr[4];

D. A standalone char b[4]; array

Correct Answer: A

EXPLANATION

This union allows writing to the int member and reading individual bytes via the array member, revealing endianness.

Q103 | Application-oriented | Unions

APPLICATIONS

Array members inside unions are commonly applied in:

A. High-level GUI design

B. Low-level data conversion and byte manipulation tasks, such as handling network byte order

C. Recursive function calls

D. String sorting algorithms

Correct Answer: B

EXPLANATION

This pattern is widely used in systems and network programming for byte-level data manipulation.

Q104 | Previous-Year Pattern | Unions

DECLARATION

In `union U { float f; char bytes[4]; };`, assuming float is 4 bytes, what is `sizeof(union U)`?

A. 8 bytes

B. 4 bytes

C. 1 byte

D. 16 bytes

Correct Answer: B

EXPLANATION

Both members are 4 bytes in size, so the union's size is 4 bytes (the maximum among equal-sized members).

Q105 | Difficult/Tricky | Unions

MEMORY BEHAVIOUR

In `union U { int i; char b[6]; };`, if int is 4 bytes and the char array has 6 elements, what is `sizeof(union U)`?

A. 4 bytes

B. 10 bytes

C. 6 bytes

D. 2 bytes

Correct Answer: C

EXPLANATION

The union's size equals its largest member; here the 6-byte char array is larger than the 4-byte int.

CHAPTER 8 OF 10 • UNIONS

Concept of Memory Saving and Union

This chapter explores the shared memory concept, memory allocation, and memory efficiency advantages that unions provide over other data structures.

Q106 | Basic | Unions

MEMORY EFFICIENCY

Why are unions considered memory-efficient compared to structures?

Union memory layout: **int i** **float f** **char c** ← *all members overlap the same block, sized to the largest membe*

A. All union members share one common memory location instead of each having separate memory

B. Unions store more members than structures

C. Unions do not allow initialization at all

D. Unions are always smaller regardless of member types

Correct Answer: A

EXPLANATION

Sharing memory among members is the fundamental reason unions save memory compared to structures.

Q107 | Definition-based | Unions

SHARED MEMORY CONCEPT

The "memory saving" concept in unions refers to:

A. Compressing data using algorithms

B. Deleting unused variables automatically

C. Reducing the number of variables in a program

D. Allocating a single shared memory block sized to the largest member, instead of separate blocks for each member

Correct Answer: D

EXPLANATION

This directly describes how unions achieve memory efficiency through shared storage.

Q108 | Concept-based | Unions MEMORY ALLOCATION

If a union has members of sizes 2, 4 and 8 bytes, how much total memory does the union occupy (ignoring padding)?

A. 8 bytes
B. 14 bytes
C. 2 bytes
D. 4 bytes

Correct Answer: A

EXPLANATION
The union occupies memory equal to its largest member, which is 8 bytes here.

Q109 | Syntax-based | Unions MEMORY EFFICIENCY

Which operator is used to determine how much memory a union occupies?

A. memof()
B. sizeval()
C. sizeof()
D. length()

Correct Answer: C

EXPLANATION
The sizeof() operator returns the number of bytes occupied by a union or any other data type.

Q110 | Statement-based | Unions ADVANTAGES

Which statement correctly explains the memory-saving advantage of unions?

A. Unions save memory by compressing all member values into fewer bits
B. Unions save memory by reusing the same memory location for all members instead of allocating separately
C. Unions save memory only when using pointers
D. Unions save memory by deleting unused data types

Correct Answer: B

EXPLANATION
This is the correct explanation: memory reuse rather than compression is the mechanism.

Q111 | Assertion-Reason | Unions

ADVANTAGES

Assertion (A): Using a union instead of a structure can significantly reduce memory usage when only one field is needed at a time.
Reason (R): A union's size depends only on its largest member, not the sum of all members.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are false

D. Both A and R are true, R is the correct explanation of A

Correct Answer: D

EXPLANATION

Both statements are true, and R correctly explains why unions reduce memory usage in this scenario.

Q112 | Match-the-Following | Unions

SHARED MEMORY CONCEPT

Match the item with its description:
I. Structure II. Union III. sizeof(largest member)
a. Determines union size b. Sum of member sizes c. Shared memory concept

A. I-c, II-a, III-b

B. I-b, II-c, III-a

C. I-a, II-b, III-c

D. I-b, II-a, III-c

Correct Answer: B

EXPLANATION

Structure size is the sum of member sizes; union embodies the shared memory concept; its size equals the largest member.

Q113 | Comparison-based | Unions

MEMORY EFFICIENCY

Compared to declaring three separate variables (int, float, char), how does using a union of the same three types affect memory usage?

A. It uses memory equal to the largest of the three types instead of the sum of all three

B. It uses more memory than three separate variables

C. It uses exactly the same memory as three separate variables

D. It uses zero memory

Correct Answer: A

EXPLANATION

A union uses only as much memory as its largest member, unlike separate variables which each need their own memory.

Q114 | Code-snippet | Unions

MEMORY ALLOCATION

```
union Data { int i; float f; double d; };
printf("%lu", sizeof(union Data));
```

Assuming int=4, float=4, double=8 bytes, what is the output?

A. 16

B. 8

C. 4

D. 12

Correct Answer: B

EXPLANATION

The union's size equals its largest member, double, which is 8 bytes.

Q115 | Output-based | Unions

MEMORY ALLOCATION

```
struct S { int i; float f; double d; };
union U { int i; float f; double d; };
printf("%d", sizeof(struct S) > sizeof(union U));
```

Assuming standard sizes, what is the output?

A. 1

B. 0

C. -1

D. Compilation error

Correct Answer: A

EXPLANATION

Since the structure sums all member sizes while the union takes only the largest, sizeof(struct S) is greater, so the comparison evaluates to 1 (true).

Q116 | Fill-in-the-Blank | Unions

MEMORY EFFICIENCY

The memory saved by using a union instead of a structure with the same members is approximately equal to the _____ of the sizes of the members other than the largest one.

A. sum

B. product

C. average

D. difference of squares

Correct Answer: A

EXPLANATION

The saved memory roughly corresponds to the combined sizes of the members other than the largest one.

Q117 | Scenario-based | Unions APPLICATIONS

An embedded system with very limited RAM needs to store one of five possible sensor value types at a time. Which approach minimizes memory usage?

A. Declaring five separate variables

B. Using a structure with five fields

C. Using five separate arrays

D. Using a union with five members

Correct Answer: D

EXPLANATION

A union minimizes memory by allocating only enough space for the largest of the five types.

Q118 | Application-oriented | Unions APPLICATIONS

The memory-saving property of unions is particularly valuable in which type of applications?

A. Applications with abundant memory and no constraints

B. Applications that never reuse variables

C. Purely mathematical applications with no data types

D. Memory-constrained applications such as embedded systems and microcontrollers

Correct Answer: D

EXPLANATION

Memory-constrained systems benefit most from the efficient shared storage that unions provide.

Q119 | Previous-Year Pattern | Unions SHARED MEMORY CONCEPT

The key reason a union requires less memory than an equivalent structure is:

A. Unions allocate memory only once, shared among all members, sized to the largest member

B. Unions use compressed encoding internally

C. Unions do not allow floating point members

D. Unions store data in a file instead of RAM

Correct Answer: A

EXPLANATION

This directly matches the standard exam explanation for union memory efficiency.

Q120 | Difficult/Tricky | Unions MEMORY ALLOCATION

A union has members: char c (1 byte), short s (2 bytes), int i (4 bytes), and double d (8 bytes). What is the MINIMUM memory this union could require (ignoring alignment)?

A. 8 bytes

B. 1 byte

C. 15 bytes

D. 4 bytes

Correct Answer: A

EXPLANATION

The union must accommodate its largest member, double (8 bytes), so its minimum required size is 8 bytes.

CHAPTER 9 OF 10 • UNIONS

Union as a Generic Data Type

This chapter explains how unions can act as a generic/variant data type, representing multiple possible data types through shared storage, and its associated benefits and characteristics.

Q121 | Basic | Unions DEFINITION

What does it mean to describe a union as a "generic data type"?

A. It can only store integer values

B. It works only with generic templates in C++

C. It cannot store any value at all

D. It can represent multiple different data types using a single, shared storage location, chosen at runtime

Correct Answer: D

EXPLANATION

A union's ability to interpret the same memory as different types makes it a flexible, generic container.

Q122 | Definition-based | Unions DEFINITION

Which best defines the "generic" characteristic of a union?

A. A union restricts a variable to exactly one fixed data type

B. A union allows a single variable to represent different data types at different times using shared memory

C. A union is generic because it works only with pointers

D. A union always behaves like an array

Correct Answer: B

EXPLANATION

This flexibility to hold varying types over time, using the same memory, is what makes unions generic.

Q123 | Concept-based | Unions USAGE

Why is a union particularly useful for implementing a variable that can hold different types across different function calls?

A. Because it wastes memory to store multiple copies

B. Because it reuses the same memory location, allowing reinterpretation as different types without separate storage

C. Because it always defaults to an integer type

D. Because it forbids reinterpretation of data

Correct Answer: B

EXPLANATION

Memory reuse enables the same union variable to represent different types as needed, supporting generic behaviour.

Q124 | Syntax-based | Unions USAGE

In implementing a generic union type for multiple data kinds, which construct usually accompanies the union to track the current active type?

A. A pointer to void only

B. A second identical union

C. A tag/type enum or int field, often within an enclosing structure

D. A macro definition only

Correct Answer: C

EXPLANATION

A tag field (often an enum) paired with the union is the standard way to track its currently active type.

Q125 | Statement-based | Unions CHARACTERISTICS

Which statement about unions as generic data types is TRUE?

A. A union can hold one of several types at a time, and the programmer must track which type is currently valid

B. A union can safely hold and validly read all its members simultaneously

C. Unions automatically convert data types without programmer tracking

D. Unions cannot be used for generic data representation

Correct Answer: A

EXPLANATION

Correct usage requires the programmer to track the currently valid member, since only one can be valid at a time.

Q126 | Assertion-Reason | Unions DEFINITION

Assertion (A): Unions are often used to implement "variant" types capable of representing multiple kinds of data generically.
Reason (R): A union allows different data types to occupy the same memory space at different times.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are true, R is the correct explanation of A

D. Both A and R are false

Correct Answer: C

EXPLANATION

Both are true, and R correctly explains the generic/variant capability of unions.

Q127 | Match-the-Following | Unions CHARACTERISTICS

Match the term with its description:
I. Generic union II. Tag field III. Variant record
a. Structure combining union and tag b. Type indicator for active union member
c. Data type able to hold multiple representations

A. I-a, II-c, III-b

B. I-b, II-a, III-c

C. I-c, II-b, III-a

D. I-c, II-a, III-b

Correct Answer: C

EXPLANATION

I is the generic capability, II is the tag field's role, III is the resulting combined pattern.

Q128 | Comparison-based | Unions BENEFITS

How does a "generic union" differ from a generic template in a language like C++ (in terms of core mechanism)?

A. A union achieves generic behaviour via shared memory reinterpretation at runtime; templates achieve it via compile-time type substitution

B. They are exactly the same mechanism

C. Unions are only usable at compile-time

D. Templates also require shared memory reinterpretation

Correct Answer: A

EXPLANATION

These are fundamentally different mechanisms for achieving flexibility with types.

Q129 | Code-snippet | Unions

APPLICATIONS

```
union Value { int i; float f; char c; };
enum Type { INT, FLOAT, CHAR };
struct Variant { enum Type type; union Value val; };
struct Variant v;
v.type = FLOAT;
v.val.f = 3.14;
```

What does this code pattern demonstrate?

A. A fixed single-type variable

B. A generic/variant type using a union with a tag to track the active member

C. An array of unions

D. A memory leak

Correct Answer: B

EXPLANATION

This is a textbook example of implementing a generic/variant type using a tagged union.

Q130 | Output-based | Unions

USAGE

```
union Value { int i; float f; };
union Value v;
v.i = 10;
printf("%d", v.i);
v.f = 2.5;
printf(" %.1f", v.f);
```

What is the output?

A. 10 10.0

B. Garbage values

C. Compilation error

D. 10 2.5

Correct Answer: D

EXPLANATION

Each print statement occurs immediately after the corresponding assignment, so both values print correctly in sequence.

Q131 | Fill-in-the-Blank | Unions DEFINITION

A union used to represent multiple possible data types in a single variable, combined with a type indicator, is often called a _____ type.

A. static
B. tagged/variant
C. constant
D. volatile

Correct Answer: B

EXPLANATION
This combination is standard terminology known as a "tagged union" or "variant type".

Q132 | Scenario-based | Unions APPLICATIONS

A configuration parser needs to store setting values that could be integers, floats, or strings, and must know which type each value currently holds. Which design is most appropriate?

A. A single int variable used for all settings
B. Three completely separate unrelated variables per setting
C. A structure with a tag field and a union holding int, float, or string data
D. An array of unrelated types with no tracking

Correct Answer: C

EXPLANATION
This tagged-union design is the standard generic approach for representing heterogeneous configuration values.

Q133 | Application-oriented | Unions APPLICATIONS

Generic unions (often as tagged unions) are widely used in:

A. Simple arithmetic operations only
B. Interpreters, compilers, and parsers to represent tokens or values of varying types
C. Fixed-size array declarations
D. String literal storage exclusively

Correct Answer: B

EXPLANATION
Tagged unions are a foundational technique in interpreters and compilers for representing heterogeneous values.

Q134 | Previous-Year Pattern | Unions BENEFITS

The main benefit of using a union as a generic data type is:

A. It permanently fixes a variable to one data type

B. It eliminates the need for any data types in a program

C. It automatically sorts data

D. It allows a single variable to flexibly represent multiple data types while saving memory

Correct Answer: D

EXPLANATION

This captures both the flexibility and memory efficiency benefits of using unions generically.

Q135 | Difficult/Tricky | Unions CHARACTERISTICS

When using a union as a generic type WITHOUT a tag field, what is the primary risk?

A. The program will simply fail to compile

B. The union will automatically throw a runtime exception

C. The union will silently convert all types to int

D. The programmer may read the wrong member, leading to incorrect or undefined interpretation of the stored data

Correct Answer: D

EXPLANATION

Without a tag to track which member is valid, reading the wrong member yields incorrect or undefined results.

CHAPTER 10 OF 10 • UNIONS

Concept of Anonymous Union

This chapter covers the definition, syntax, member access, characteristics, applications and advantages of anonymous unions in C.

Q136 | Basic | Unions DEFINITION

What is an anonymous union?

A. A union that cannot be initialized at all

B. A union declared without a tag name and without a variable name, whose members are accessed directly

C. A union with only one member

D. A union declared inside a comment

Correct Answer: B

EXPLANATION

An anonymous union has no tag and no variable name; its members become directly accessible in the enclosing scope.

Q137 | Definition-based | Unions SYNTAX

Which of the following best describes the syntax of an anonymous union?

A. union Tag { int a; float b; } u;

B. union { int a; float b; };

C. union { int a; float b; } u;

D. typedef union { int a; } U;

Correct Answer: B

EXPLANATION

An anonymous union omits both the tag and the variable name, as shown in this option.

Q138 | Concept-based | Unions MEMBER ACCESS

How are members of an anonymous union accessed?

A. Directly by their member names, without any variable name or dot operator, since they behave as if declared in the enclosing scope

B. Using a variable name together with the dot operator

C. Using array indexing

D. They cannot be accessed at all

Correct Answer: A

EXPLANATION

Because there is no variable name, members of an anonymous union are accessed directly by name in the enclosing scope.

Q139 | Syntax-based | Unions SYNTAX

Which of the following is a valid anonymous union declaration inside a structure?

A. struct S { int type; union { int i; float f; }; };

B. struct S { int type; union U { int i; float f; }; };

C. struct S { int type; union { int i; float f; } u; };

D. struct S { union int i; };

Correct Answer: A

EXPLANATION

This option shows a correctly declared anonymous union nested inside a structure, without a tag or variable name.

Q140 | Statement-based | Unions CHARACTERISTICS

Which statement about anonymous unions is TRUE?

A. Anonymous unions allow direct access to their members without a union variable name

B. Anonymous unions must always have a tag name

C. Anonymous unions cannot be nested inside structures

D. Anonymous unions require pointer access only

Correct Answer: A

EXPLANATION

This is the defining characteristic that distinguishes anonymous unions from regular named unions.

Q141 | Assertion-Reason | Unions

ADVANTAGES

Assertion (A): Anonymous unions simplify member access by removing the need for an intermediate variable name.
Reason (R): Members of an anonymous union are treated as if they belong directly to the enclosing scope.

A. Both A and R are true, R is not the correct explanation of A

B. A is false, R is true

C. Both A and R are true, R is the correct explanation of A

D. Both A and R are false

Correct Answer: C

EXPLANATION

Both statements are true, and R correctly explains why anonymous unions simplify access syntax.

Q142 | Match-the-Following | Unions

SYNTAX

Match the declaration with its type:
I. `union { int a; } b;` **II.** `union { int a; };` **III.** `union Tag { int a; } t;`
a. Named union with a variable name (not fully anonymous) b. Anonymous union c. Fully named/tagged union

A. I-a, II-b, III-c

B. I-b, II-a, III-c

C. I-c, II-b, III-a

D. I-a, II-c, III-b

Correct Answer: A

EXPLANATION

I is a union with a variable but no tag (not fully anonymous), II is truly anonymous, III is a fully named union.

Q143 | Comparison-based | Unions

MEMBER ACCESS

How does an anonymous union differ from a regular (named) union in terms of member access?

A. There is no difference between them at all

B. Regular unions cannot be accessed at all

C. Anonymous unions require pointers for access

D. Anonymous union members are accessed directly by name; regular union members require a variable name and the dot operator

Correct Answer: D

EXPLANATION

This access-syntax simplification is the key practical difference between anonymous and regular unions.

Q144 | Code-snippet | Unions

APPLICATIONS

```
struct Point {
    int type;
    union {
        int x;
        float fx;
    };
};
struct Point p;
p.x = 10;
printf("%d", p.x);
```

What is the output?

A. 10

B. Compilation error since the union has no name

C. A garbage value

D. 0

Correct Answer: A

EXPLANATION

Because the union is anonymous, its member x is accessed directly as p.x, correctly printing 10.

Q145 | Output-based | Unions

MEMBER ACCESS

```
union {
    int a;
    char b;
};
a = 65;
printf("%d %c", a, b);
```

Assuming this anonymous union is used validly in its context, what is the likely output?

A. 65 A

B. 65 65

C. A 65

D. Compilation error in every context

Correct Answer: A

EXPLANATION

Member a set to 65 overlaps with b; since 65 corresponds to 'A' in ASCII, printing b as %c shows 'A'.

Q146 | Fill-in-the-Blank | Unions DEFINITION

An anonymous union is declared without a _____ and without a _____.

A. semicolon, brace

B. tag name, variable name

C. member, type

D. function, loop

Correct Answer: B

EXPLANATION

The defining feature of an anonymous union is the absence of both a tag name and a variable name.

Q147 | Scenario-based | Unions APPLICATIONS

A programmer wants to embed a union inside a structure but avoid using an extra variable name each time a member is accessed, for cleaner code. Which technique should be used?

A. A regular named union

B. An anonymous union

C. An array of unions

D. A separate global union

Correct Answer: B

EXPLANATION

Anonymous unions are specifically designed to simplify access by removing the need for an intermediate variable name.

Q148 | Application-oriented | Unions APPLICATIONS

Anonymous unions are commonly applied in:

A. Declaring global constants

B. Simplifying access to variant fields within structures, such as in graphics or vector libraries

C. Implementing recursive functions

D. String formatting operations

Correct Answer: B

EXPLANATION

Anonymous unions are frequently used in libraries (e.g. vector/graphics math) to allow flexible, direct field access.

Q149 | Previous-Year Pattern | Unions ADVANTAGES

A key advantage of using anonymous unions within structures is:

A. They allow direct, simplified access to union members without an extra intermediate name

B. They increase the number of required variable names

C. They prevent any member from being accessed

D. They convert the structure into an array

Correct Answer: A

EXPLANATION

This directly matches the standard exam explanation of the anonymous union's main benefit.

Q150 | Difficult/Tricky | Unions MEMBER ACCESS

Consider a structure with two anonymous unions, each containing a member named `value`. What issue arises?

A. A naming conflict/ambiguity occurs because both members would be accessed directly by the same name in the same enclosing scope

B. No issue at all; anonymous unions never conflict

C. The compiler automatically renames them without error

D. Anonymous unions cannot have member names at all

Correct Answer: A

EXPLANATION

Since anonymous union members merge into the enclosing scope, two members with identical names would cause a naming conflict.